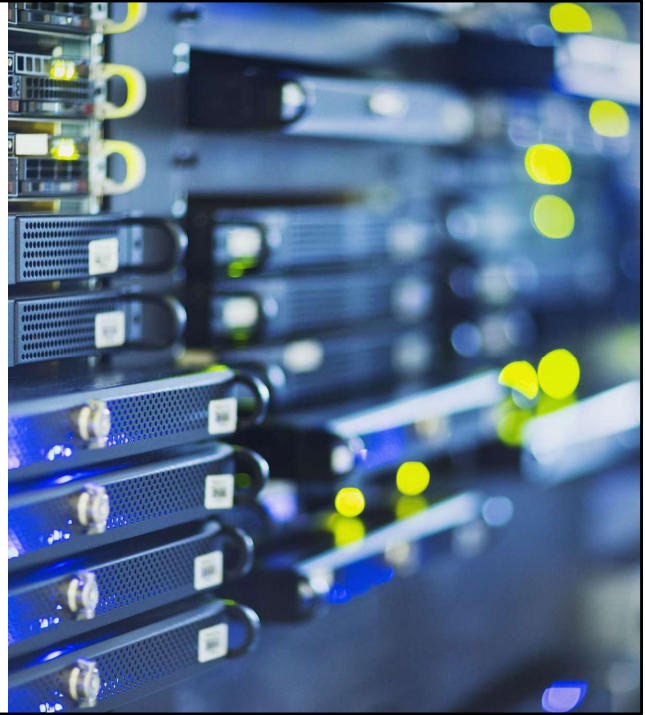




The majority of you will probably end up using will data mining.
Which here we define as taking existing data that's out there and doing cool stuff with it.

Data Availability

- No shortage of data in today's world
- But that comes with numerous challenges:
 - Dealing with big data
 - Selecting appropriate data
 - Cleaning and processing data
 - Extracting useful insights
- Other courses cover most of these challenges, so for now, let's focus on selecting appropriate data



There's obviously no shortage of data in today's world.

Every day we're generating up to terabytes of new data now. Not all that data is useful. Not all of it's really all that meaningful, but there is data out there.

And often, large quantities that can be useful in answering certain questions and understanding a lot of real world applications and real world behaviors and systems.

The availability of data, though, does come with its own challenges; one of those is just dealing with large data sets.

As you need the technical systems in place in order to both store and process large data.

We'll talk a little bit about working with big data as we probably go along, and then you'll hear a bit more in 699, but we will mainly focus on these last three points, which is with so much data out there, the key skill is to select the appropriate data.

It's easy to just throw data at a problem.

And yes, with enough data that might get you some good results, but that comes with its own tradeoffs.

But that comes with its own tradeoffs.

Obviously you have to have the resources just to handle all of the hardware and

energy demands.

For many problems, smaller, higher quality datasets could be curated to build ML/AI models in more efficient, sustainable ways.

Rather than ill-defined general models, models built for specific tasks with more robust testing can be more useful.

Curating the appropriate data and figure out how much of it you need and what you need to measure. That is a skill and not always the easiest.

It can be technically challenging and resource intensive to carefully curate a data set compared to just take whatever data you can get easily.

Once you have the data, there is cleaning and processing the data.

The vast majority of the data that's out there nowadays is unstructured, in formats that are not easily quantifiable or easily analyzable.

So in things like.

Structured data.

It's like Jason's and SQL formats could be non numeric data like text and images.

Could be a lot of time series data that often is not necessarily the cleanest or the most readily useful for for analysis, so lots of.

Issues and each kind of type of data will have its own recommendations and.

Best practices involving cleaning and processing, that's another thing you need to think about and that will be part of what you actually write about in your methods is, you know, given what your problem is and what your data is, what are the most appropriate cleaning and preprocessing steps.

So some of you have taken the NLP classes where we talked about, you know.

Some people approach NLP and think that you can just apply the same preprocessing and cleaning steps.

To every single case, and that is just a checklist, and that's not a good way to think about it.

That shows you don't really understand how text works or how analyzing it works.

You know you really have to think about for whatever your problem is, whatever techniques you're going to be applying.

What's the most appropriate pre processing kind of pipeline to appropriately?

Handle the data for that problem.

And then the kind of big thing is all of this is so that we can actually extract useful insights from data because.

There's with the data that is being constantly generated by all our apps and all our devices, and you know all the organizations and systems.

A lot of it's noise and a lot of it's garbage, so finding the signals and finding the the right useful patterns.

And insights can be a struggle. So.

That's one of the things that you will really want to think about as your developing your methods for your capstone project is, you know, what is the the signal you're looking for in the data.

What is it that you're trying to extract?

Now, for tonight at least, we're gonna focus on selecting the appropriate data.

We'll kind of hit a little bit of the other stuff, but mostly those are covered in other courses.

Features of Available Data

Public versus Private

Structured versus Unstructured versus Semi-structured

Sparse versus Dense

Raw versus Transformed

Well-documented versus Poorly Documented

So when it comes to kind of categorizing or systematizing.

The data that is available, there's a lot of features to consider.

In terms of like the most practical one for you to consider with your capstone project is do you have access to it?

So is it a public data set or is a private data set because a lot of the the data sets out there and a lot of the most interesting data sets?

Are being generated are collected by companies who? That's their proprietary data. They're not gonna want to share that necessarily.

But there is also a lot of public data sets out there, either a lot of companies will modify or de identify or otherwise try to anonymize some of their data sets to put out into the public to kind of for things like hackathons to see what can be.

Done. But for the most part, you know you're going to be limited to the the data that everyone has access to.

If you do.

In talking to some of you about your projects, a few of you might have access or connections to private data, and if so, totally cool.

You just got to make sure you get all the right permissions and everyone's on board with how you need to use the data.

It's important when you're using private data for academic or other purposes that

it's not designed for, that you really.

Make sure that you have all the necessary.

Necessary consent and agreement and a plan for protecting the data and the the privacy of the data.

But most of you will be working with public data.

Next thing is you know the structure of the data set a lot of.

Well, all of analytics, every kind of quantitative thing that you can do with data does require that data be structured.

IE that data needs to be in basically numeric data frames.

And a lot of data is not already in that form.

So if you have unstructured or semi structured data you have to figure out ways in which to quantify it and those often introduce some level of.

Of error or imprecision to the the proceeding. So in the case of text and natural language processing, there are a ton of different ways that you can take words and convert them to numbers.

But every single way you want to do that will involve some information loss.

There is deep and rich and nuanced, meaning in words generally, some more than others.

It depends on the context, but in converting words.

The kind of patterns of language and to numbers. You're gonna lose some of that meaning and some of that nuance now some of the more complex.

Methods for quantifying and structuring text like the you know, the deep neural Nets.

Those have less information loss and say if you do a simple.

Word frequency analysis. But even those complex models, you're still losing information. You're still.

Sacrificing.

Some of the information that people get from language in order to make it, you know, something that's analyzable now, I I think one of the best.

Approaches to a lot of the issues with big data and various things is to really consider semi structured data.

So this generally is gonna refer to data that's in certain formats.

Jason formats what we'll talk about in a second.

That are mainly useful because one of the downsides to data frames and like you know, data.

Excel Sheets spreadsheets is.

They're not the most efficient way to store data. If you know the memory requirements can balloon pretty quickly, and so other types of data structures like Jason.

On an XML and sequel.

While you have to do some trade-offs with those.

Structures are not as easily analyzable.

They are much more efficient memory wise and still allow you to keep a lot of the same relationships and structures of data.

But whether or not you use that really depends on what data you're working with and how people stored it.

Next kind of two things come to are related to each other so.

You your data can be sparse at or could be dense, so this is mainly gonna be an issue of temporal or geospatial data.

So one of the big things we often want to look at is forecasting over time.

Or you know otherwise understanding temporal patterns and.

For some problems, sometimes the data that's easier than others 'cause there's a lot of time series data.

Where the data points are nice and regular and maybe even, you know represent.

Second, by second or minute by minute measurements, whereas other time based data might be a little less frequent or a little less.

Regular.

So when I worked a lot with social media data, you know, while people were pretty much always tweeting.

They weren't necessarily doing so regularly.

It's not, you know, I guess some people schedule their their social media posts, but.

The time between post could vary quite a bit.

You know some people.

Post every day, every several times a day.

Others might post once a year, and so that sparsity of that data can create its own issues, but often represents.

Is a little bit more realistic, whereas you know if you're working with stock market values, those are nice ones. They're dense. And two, the regular you have very little missing data.

In you know stock.

Returns.

So it's not one is not better than the other.

They just come with their own challenges that you want to consider because if you want to do forecasting, you know time models.

You do often need dense regular data.

Sparse data won't really work, but there are ways around that.

There's other types of methods you can work with with sparse data.

And then raw versus transformed so.

This is.

Is a big issue with unstructured data, because obviously that has to be transformed in some way.

But even other data sets you know?

The.

The numbers you see in the data frame may not be the original numbers for several different reasons.

So sometimes data is transformed so it's modified somehow in order to anonymize the data, so it could be that the original data was way too personable, personally identifiable.

And so before it got made public, you know, that was transformed. So it was. Less identifiable so.

Sometimes this is done with demographic information where maybe originally the data set had, like a specific birth date for a person which is not super identifiable, but it's not great to have that just publicly out there. So that might get transformed into an age range.

And while not a problem necessarily, is important to note those those shifts and changes in the data, because it does impact the interpretation.

Sometimes Data's transformed because the transformation makes more sense mathematically.

Like you know, maybe the original.

Form of the data.

Was kind of a frequency count that might not be very normally distributed.

It and so it got, you know, normalized to account for frequency or population values.

Again, not a problem.

And often you would do that anyway as part of the analysis, but you want to note that and make sure it's documented.

And then the final thing is what I really wanna focus on and what I'm really.

What? I'm gonna judge your data source selections on.

If you're getting.

Available data.

It has to be well documented.

And that means that there has to be information tied to the data about where it came from, how it was collected, when it was collected.

Any inclusion exclusion criteria used for the data collection.

Any transformations done to the data?

As much information about the source of the data as possible.

Because one of the kind of downsides to a lot of the database and data archives like Kaggle is anyone can post to it.

And while that is also kind of a good thing, and you know terms of democratization of information, also a bad thing, because there's no necessary checks in terms of is it?

Reliable source of data.

Is it a meaningful data set?

And so when you go on cable and you see perfect data set and you click on it and it has the data set for like no other information.

That is not data that's gonna be useful.

Even if it is, you know, legitimate quality data if no one took the time to explain that and to give evidence for it being.

Good quality.

Data. It's not something you should be using.

Even in the case of so talk to a few of you about synthetic data and when you can and can't use that.

And another thing to watch out for on these database science is often people will post synthetic data which could be useful in limited cases, but synthetic data is not going to be useful in getting you insights about the.

That that data is about like you can't use that to get insights and make recommendations about how people are gonna behave or, you know what might be impactful to people because at the end of the day, it's fake data.

It doesn't actually represent people's attitudes, behaviors, beliefs, whatever.

It is good at testing out math and algorithms.

Good at, you know, trying to figure out the boundaries and the kind of assumptions being made about.

The patterns and the data and the distributions of the data.

And to kind of pilot and build certain tools, but it at the end of the day, it's not real data.

Not actually a good representation necessarily.

Of whatever it is you're studying in the real world.

So that's the one.

The other four things you know, it all depends on what it is you're working on and what's available.

But this last thing, that's where we're really gonna judge whether or not data you find or data you can collect is good.

Common Strategies for Getting Data



APIs (application programming interface)



Databases (SQL, MongoDB)



Web Scraping



Archives and Repositories

Now some common strategies for getting data.

The easiest one's gonna be the archives and repositories of finding data set datasets that others have already kind of constructed and built.

But that's not always going to be possible.

The kind of next level would be databases.

So those are going to be systems for storing data, but generally speaking databases are used in private sector data.

There are a few public databases that use this architecture, but not as commonly.

So sometimes you might be left with getting data through either an API, which is basically just a system for connecting to.

To an app or to a website or web scraping.

So just getting data from on the web now.

If you're really interested in these.

These things.

The one of the natural language processing classes annually 540. We actually go into and have demos of how you use some of these things.

Especially because it is especially web scraping.

Is one of the best ways of getting text data and other unstructured data.

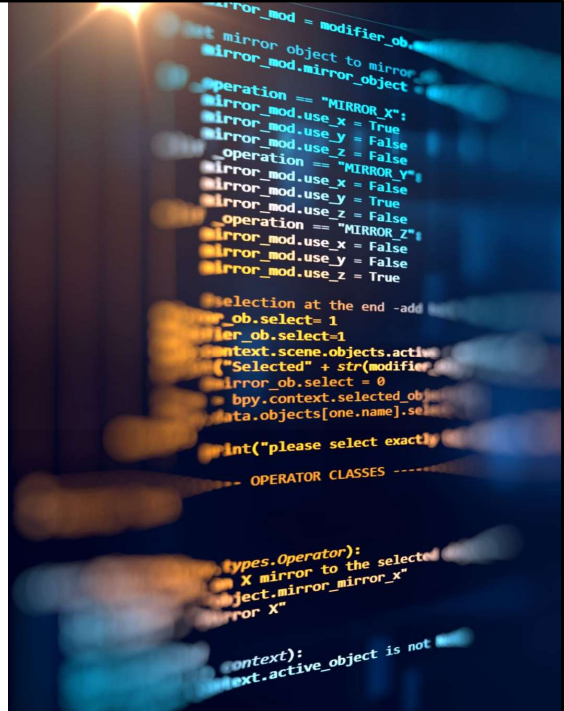
Kind of.

But I will kind of briefly go over the steps in the processes and I have some links to some resources.

So if you wanna learn more, but let's start with APIs so.

APIs

- Used by many websites and apps to allow developers and others access
- Common APIs for research: [Reddit](#), [IMdB](#), [YouTube](#) (limited), [WorldBank](#), [Yelp](#) (not free), [Bluesky](#)
 - [List of Public APIs](#)
- Vary in terms of type and level of access
 - Many require special permissions and have restrictions on use
- Many APIs have specific Python packages with custom functions
 - Alternatively, requests package can be used for any API
- Common types of APIs: REST and GraphQL



API's were developed.

For software developers.

They were built to allow developers to have access and connections to websites and apps so that they could, you know, automate processes.

Or control information flow, things like that.

A secondary use of AP is though, is data collection.

Because the connection is not one way.

It's not just a matter of pushing information or data to the source, but you can also get data from the source.

Most of the time it depends on the API and what the permissions are.

Now.

Five years ago, seven years ago.

The amount and access to AP is was a lot more expansive than it is today.

In the last few years, especially social media sites have really locked down. Their AP is and access.

The researchers, for good and bad reasons on the good side.

They, you know, became a bit more concerned with the privacy of their users and ensuring you know that.

People weren't collecting data that could be tied to individuals on the bad side.

Most of them actually did it so they could make money off of it, like Twitter used to have. One of the best AP is and the best access for academic research.

But when a certain someone bought it.

And pay too much money for it. Decided that the way he can make his money back was by charging people to have access to the API and for getting data.

So Twitter's out out of most people's price ranges now, but there are plenty of others that are still kind of existing now, not all of them are free, and not all of them are completely opened like YouTube has an API where you're able to get video.

Comments and some other sources of data, but you can't get everything and there are some limits as to how much you can get.

Generally added time.

Others are a bit more.

Open Reddit.

It depends on the day they go back and forth with how open.

And expansive their API access is, but generally speaking greed is pretty easy to get and to get quite a bit of IMDb also is pretty open for getting data.

The newest one that's really interesting is Blue Sky.

So Blue Sky is the kind of new alternative.

To Twitter.

Actually built by the original.

One of the founders of Twitter.

And has pretty much the exact same interface and like design, it just has some different background architecture. I think it's still not the size of Twitter, but it's been pretty rapidly growing, especially in the last half a year.

Think so far?

It's a lot of academics.

And.

Intellectual on it.

But imagine that will will shift.

So plenty of these out there in the lecture notes, they have a GitHub link that links to a list of public APIs.

Now note that just because it's a public API doesn't mean you can actually get data from it.

It just means that anyone can request a developer account.

Exactly what?

You can use the API for and what you can get. That's gonna depend on the the API access.

Some. So there's a a big amount of variance in terms of the level of access for different types of APIs. Some of them require like an approval process to get a developer account, but then once you're approved for that, you have pretty.

Free rein on the data.

Others.

Any collection of data requires approval ahead of time.

Like Red, depending on the day will sometimes require you to tell them exactly what you're wanting to collect and why before they will allow access to some of the bigger data sets.

So you really wanna pay attention to the documentation of the APIs and the all the terms of service.

They have different levels of.

Authentication as well. Some like blue sky is very basic.

You really just have to have your own blue Sky account. You can use that to access the API. Others it's like several layers of encryption and authentication before you can actually get to anything.

A lot of the bigger AP is have either specific Python packages or like wrapper packages that have custom functions for dealing with that API in specific, specifically, which does help the process cause the one of the problems of APIs which, yeah, is the data structures. It varies from.

API to API and just.

The request package in Python that allows you to.

Connect to websites and apps.

Works, but it can be a slog to figure out exactly how to get to the information you need, and these pretty expansive data structures sometimes.

Steps.

There are also a few different types of AP is there's the rest API which is by far the most common and easiest to use.

There's also something called graph QL which.

Basically it's the same function and idea.

It's just a little bit of a difference in how the data is stored and sent between systems.

I from looking at the documentation and everything.

Hypothetically, graph QL is supposed to be more efficient.

At sending data.

But it is much harder to use than the rest API's.

It comes with its own Python package. You have to use.

And it can be a lot steeper learning curve to figure out exactly how you you deal with it.

But most of this, you're just going to, you know, if you have an API for the website or app you want data from.

It's the code you write is gonna be specific to that API, but there's a lot of, you know, tutorials out there.

A lot of guides, usually when I come across a new API, do a mixture of looking at the documentation.

Some are better documented than others, like blue skies.

Pretty well documented. Like what?

Their API is and how to use it?

There's a few errors they need to fix on their documentation, but generally pretty good.

Other APIs.

Not so.

Not so much like Imdb's API is a bit confusing.

I don't think it's super well documented.

But so many people use it.

So if nothing else, there's like Stack overflow post and other like tech blogs and articles with examples and tutorials on how to use it.

And this is kind of an example of the different data structures you might.

Come across an API.

This is not an exhaustive list, but these two are the most common.

So basically the issue with AP is that you're sending data across.

AI.

Don't know virtual hindling whatever analogy you want to use there, but you're sending data between 2 apps. The app you're wanting to connect to and then your own system.

And to send data it needs to be, you know, structured and packaged in a way that's both effective and efficient.

And these two are two of the architectures that seem to do that the best. So Jason is JavaScript.

Oriented Object object notation.

Something like that. It stands for something.

But basically it is a nested type of data frame which allows you to have levels of data attributes.

With all the various.

Levels of.

Values associated with those various attributes.

So Twitter use Jason.

And at the top level, you would have the tweet itself.

Then the next level would be like the message and the user and then some stuff about replies and within each of those you know you have the message and then you would have the text of the message and the hashtags and all of that. And for the US.

You would have the user name, the user URL, the follower account, all that kind of stuff.

And you would be able to pull the specific parts of that Jason you wanted.

There's also some AP. Is that use XML which?

Is the same ish idea, but rather than having this kind of nested dictionary kind of structure.

That kind of is analogous to Python dictionaries. XML is more like HTML.
So it's more a series of nested tags, so you still have levels to it, but each attribute is associated with A tag.
Rather than kind of this level in a key value pair for Jason.
So that's the overview of AP is any questions on that before we talk about web scraping?
All right then.

Data Structures in APIs

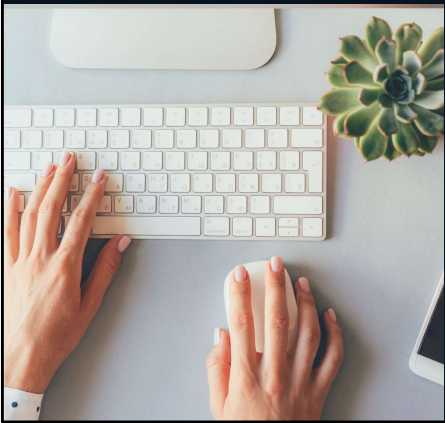
JSON

```
{
  "tweet": {
    "message": "some arrays in this tweet...",
    "tags": ["elasticsearch", "wow"],
    "lists": [
      {
        "name": "prog_list",
        "description": "programming list"
      },
      {
        "name": "cool_list",
        "description": "cool stuff list"
      }
    ]
  }
}
```

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<tweet>
  <message>some arrays in this tweet...</message>
  <tags>
    <tag>elasticsearch</tag>
    <tag>wow</tag>
  </tags>
  <lists>
    <list>
      <name>prog_list</name>
      <description>programming list</description>
    </list>
    <list>
      <name>cool_list</name>
      <description>cool stuff list</description>
    </list>
  </lists>
</tweet>
```

Web Scraping



- Method for getting content from web sites
- Common crawl methods can collect large amounts of web content but tends to be messy and unfocused
- Better approach is to be intentional with what is collected and collect semi-structured data not just completely unstructured data
- Must be careful to respect Terms of Use and privacy
 - Good rule is to limit scraping to publicly available pages not requiring a sign-in or account
- General process involves using html code to identify or generate urls then extracting needed data from html code underlying the website
 - RSS Feeds or sitemaps can be an alternative method for data extraction

Moving on to web scraping.

This is a method for basically just getting content from websites.

There are some websites where even if you technically could.

Do web scraping. It's very much.

An enormous pain to do it, and it's better to use the API.

They have access to like so.

Oh.

Hypothetically.

You could use web scraping to get tweets still.

You would be violating the terms of service and potentially get in legal trouble.

But hypothetically, it could happen.

But even if you could, because of the structure of a lot of social media sites.

It is sucked enormously inefficient.

And tedious process to try to do these things.

With those, so for social media and some of those more kind of.

Social media, like where there's a lot of.

Engagement and back and forth.

Web scraping I would not recommend, but for other things where there you know.

Pages of text or pages of information.

Or structures to information web scraping can be a very good tool, so probably one of the most common things web scraping is used for is to get news.

Either news headlines or news articles, depending on terms of use.

Are getting, you know.

Various pages from websites Wikipedia. Because Wikipedia's also has an API, but it's a garbage fire.

I could not figure that out their documentation.

But basically what this does is allows you to take advantage of.

HTML requests and HTML structure to get large amounts of data.

Now there's kind of two approaches.

The one that a lot of early.

Language models use was something called a common crawl, where basically you set you start on some webpage on the Internet.

You get all the data from that Web page, including in the background HTML that supports the web page. You'll be able to get all the web.

Web pages the URLs that that first page links to, so all the hyperlinks.

So you get all the hyperlinks from page one, you go to them, you get those pages, you get all their hyperlinks and then you keep doing that.

And that's why it's called a crawl, because it's kind of like a spider web. You just keep getting more and more as you move through the process.

Does get you large amounts of web content, but tends to be very messy.

And.

So generic and.

Unfocused that it's often not very useful.

So this is what I'm saying. You know, with the big llm's, they use something like this to basically get the entirety of language on the Internet.

And just shove that into a neural net, which.

I mean, you could see how this was a problem in the very early iterations of ChatGPT and the other models. Those early versions did not work very well.

Because.

It's just not a good way to learn language.

Yeah, been on the Internet. The Internet's a weird place.

And so.

Later versions of all the language models incorporated.

Feedback and reinforcement loops and some more targeted.

Training.

But just common crawl usually isn't super useful.

So what's a better approach is to be intentional with what you collected and collect it in a kind of semi structured data format. So not just getting text.

From websites, but getting metadata associated with that text and creating something a lot more comprehensive.

Now, like I said, you have to be careful to respect terms of use and privacy. The kind of good rule of thumb I use is that if you have to have a sign in or an account to see the web page, that's off limits for scraping, because that's probably gonna violate the terms of use.

But if it's a web page that anyone could go and access to and there's no log in required.

Generally it's up for grabs, but you do want to be a bit careful with it.

Often times, if you're trying to get something that you shouldn't be, you'll get a four.

O3 error.

Yeah, that's the forbidden 1.

The error you get when you hit, you're trying to access something you don't have access to.

And if you hit that, usually that's a good sign that maybe this isn't something you can get.

Sometimes it's not.

Sometimes it's just. I have firewalls against you.

The DoD attacks.

Whatever that thing is.

When a hacker spam something so they can shut it down.

So it does depend a bit, you know.

But usually you can Google and kind of get a sense of whether or not it's actually going to be a problem, or if it's just something you have to approach slightly differently.

So just like the common crawl, though, even if you're trying to get.

Specific set of.

Text or.

Curated corpus.

Or data set.

Or data set.

The general process just involves using HTML code to identify and generate and or generate URLs and then extracting.

The data you need from that underlying HTML code from those URLs.

There are, depending on what it is you're wanting to get and the.

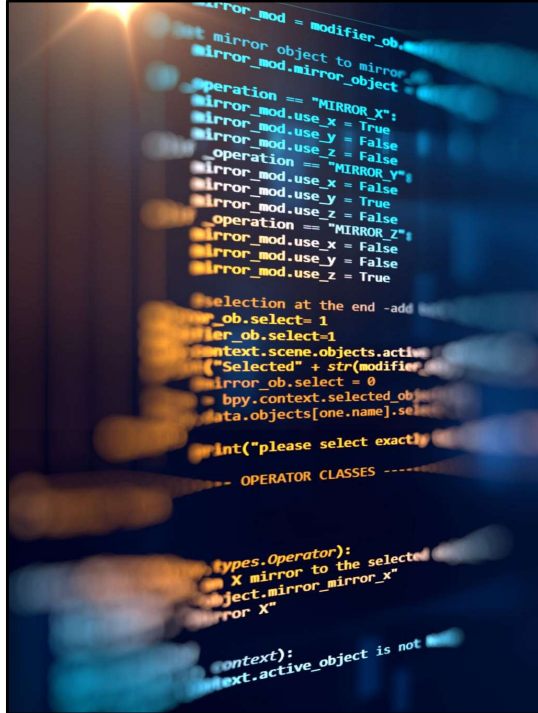
Structure of the site.

There are also ways to use RSS feeds and site maps.

To generate the URLs and to get the information needed. RSS feeds are particularly useful for.

News sites almost every news site has an RSS feed that will store basically all the.

Headlines and brief descriptions and then URLs to the full articles, all in one spot. That's really easy to extract.



Web Scraping Plan

- Before beginning web scraping, need a plan for what you want to get
 - Starting URL
 - Inclusion Criteria
 - Pattern of URLs to loop through
 - HTML pattern for extracting URLs
 - Data to collect from each page
 - HTML tags/patterns for data

In terms of the kind of how you have to go about web scraping, it depends entirely on what it is you're trying to get, but generally speaking you need to figure out you know what's your starting URL like where are you.

What are the Url's you need to define in order to get the data you need?

What are gonna be the inclusion criteria so you know and getting a lot of new sites?

Some news links will be for text based.

Stories others will be for video based stories, and generally when I'm doing this. I you can use videos, but it's a whole thing so I always generally limit to only text based articles.

And from there you can start getting the kind of patterns of URLs to loop through. So if you know you're on a web page and has pagination.

Typically speaking, those are gonna.

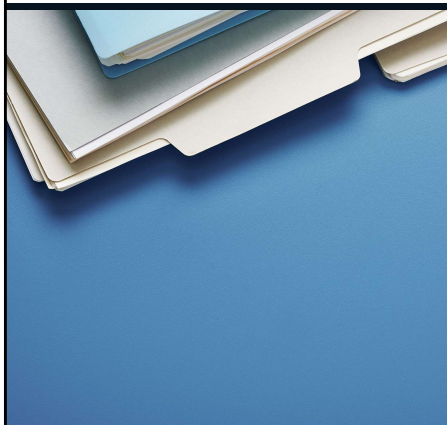
Involve having a very regular URL where you're just changing maybe the number at the end to get to the next page, and so you can loop through that pattern, then use HTML tags and patterns to get you know new URLs and to get the data you need from.

Each page using again the HTML tag tags and patterns for data.

After I get through everything, I'll actually go and show you some examples of

these different types of data and what they look like and what you want to look for.

Repositories and Archives



- Extensive data available in repositories like Github, HuggingFace, and Kaggle
 - However, much of these data is poorly documented and poor-quality data
 - Any data from these types of repositories needs to be used carefully
 - Only use datasets with data cards/descriptions documenting the source of the data and how it was collected (ideally, data is tied to a peer-reviewed publication)
 - If possible, check at least a few data points against the original source for accuracy
- Many organizations also maintain data archives that are either openly available or accessible with permission
 - These tend to be better documented and better quality depending on the organization
 - Often domain-specific datasets such as [health care](#), [economics](#), [public policy](#)
 - Many have stricter terms of use which only allow non-commercial, academic use

The easiest way to get data is with repositories and archives, so you know you're all familiar with the.

Places like GitHub and Huggins and Kaggle there are some more like specialized and domain specific repositories.

A lot of government websites actually will have data repositories.

But again, you need to make sure that you're getting.

Good data, so especially Kaggle, but even GitHub and hugging face to some extent.

A lot of people just post datasets on those without any supporting documentation, without any kind of evidence, and it'll just be, you know, CSV file. So you need to use these carefully.

All of so hugging KAF and Kaygel both have data cards.

Associated with each data.

Now, sometimes people don't fill out the data cards and it's blank, but you're looking for data that has a well developed data card with all that documentation on the source and data, and while it's collected GitHub, you want repositories that have you know.

Readme files that are clearly and amekily described what's in the.

The repo and how the data was collected.

Sometimes you'll get lucky and you'll find data that's actually tied to a peer review publication.

Because increasingly in the last decade or so, big push in academic research to make data publicly available for published research.

And so a lot of academics who are doing, you know, research in various domains. Will, when they publish their findings, will also publish the data sets and code that they were working with as well.

Beyond, you know, making sure you're only getting data that has.

You know good supporting documentation.

It's if possible. It's also good to check some of the data points against the original source. So like you know, if you find a.

Data set on kaggol that's stock values over the last five years.

Go check other sources that report stock values and just you know 5-10 data points.

The other numbers match up. Or is there some discrepancies?

You're just basically looking to see, you know is there.

Good reason to believe that the data's mostly accurate.

Or are there bigger problems?

And again, there are other data archives out there that are either openly available or accessible with permissions, like there's a couple that I sometimes use that are survey based.

The ****, the American National election studies.

Where it's a repository of.

Every other year.

Very large, nationally representative survey of Americans about politics and elections. And it is not completely open, but you can apply for access and they pretty much will approve all the academic uses of the data.

And then for specific domains, there might be repositories like for healthcare the CDC.

The NIH.

At least until recently.

Extensive repositories and archives of relevant data sets from public health studies believe most of them are still there, but.

So that if that's the domain you're working with, you might check there for other domains. There's other archives that have some links here for economics and public policy.

But.

I make these are generally gonna have a stricter terms of use than you know, hugging fans and cable.

But.

They should allow for noncommercial academic use.

A lot of them just restrict commercial use.

Databases



For many private, offline datasets, data may be stored in a database structure like SQL, MongoDB



Key to working with databases is having clear documentation of the database structure and knowing what data to select and extract from the database



[Key Terminology](#)



[Coding Example](#)



Similar to dataframes or JSON just with different syntax and memory storage

There are some other courses that you can take if you're interested in learning about.

Database structures, but a lot of you know the private datasets.

Might the data might be stored in these types of structures like SQL or Mongo DB?

The key here is making sure that whoever built this.

Properly documented what they did.

And documented what the structure is and what you know. All the values are that you need.

To collect that.

Have some links here if you wanna know. You know get the terminology of these around and a brief coding example as well to use with SQL.

I've only ever done worked with SQL myself, but it's pretty straightforward I would say.

Very similar to working with Pandas dataframes, just you know little different in terms of the syntax.

But otherwise you know very comparable.

If you're familiar with working with either data frames or Jason or other data architecture.



Let's Explore Data Availability on Topics of Interest

Optional Example to run through

Problem Definition

- Regardless of what you are doing, you must start with a clear definition of what you want/need from the research
- You can't judge the appropriateness of the data or models without knowing what you need from the data and analysis
- Problem definition can evolve and change however given data availability or other resource constraints
- Often more than one approach/dataset for a problem
- If an ideal dataset is not available, multiple datasets may need to be used or combined to solve the problem

Practical Example: AI Solutions

- Many want to 'AI' solutions to problems but do not define what the problem or desired solution is
- AI is a broad term that can encompass many approaches, and the needed complexity of the model depends on the output needed
- To determine appropriate data needed, should define:
 - Outputs needed (insights, predictions, novel generations)
 - Potential inputs (structured data, text, images, etc.)
 - Existing solutions
 - Scope of the solution (narrow, general)
- Clear, thorough evaluation of needs and limitations is essential for responsible, ethical research

Practical Application: Sentiment Analysis

- A widely used NLP technique for measuring 'opinion' in text
- Recently, some push for AI-based sentiment analysis
- But depending on the application and approach, this could be overkill and a poor solution
- Need to clearly define problem and understand existing approaches:
 - What output is needed? (binary classification, continuous prediction, distinct 'feelings')
 - What domain(s) matter? ('general purpose', finance, health, etc)
 - For the domain, how easy is sentiment to identify?
 - Do any existing approaches or pre-trained model fit the domain and output?
 - What data is needed?